

Step 2: Entity Description

Question

The *Question* entity represents the trivia questions from our dataset. A question has a *questionID*, a *questionType* ("multiple-choice" or "true-false"), and *questionText* (the question itself, e.g. What is the closest planet to Earth?).

We modeled this as an entity rather than an attribute, since each question has its own associated data fields specific to itself. In addition, *Question* has relationships to multiple other entities, and modeling it as an attribute of another entity would not capture these relationships.

QuestionOption

The *QuestionOption* entity represents the options for a *question*. A QuestionOption has a *questionOptionID*, a *optionLabel* ("A", "B", "C", "D", "True", "False"), and *optionValue* ("Earth", "Mars", NULL for T/F questions, etc.)

We modeled *QuestionOption* as an entity since each question has multiple options. Because a question has multiple options and the number of options is variable (depending on question type), it does not make sense to have the option as an attribute of *Question*.

Category

The *Category* entity represents the category of a *question*. A Category has a *categoryId*, and a *categoryName*. We modeled *Category* as an entity rather than an attribute, since each game can have a variable number of categories associated with it and attributes cannot be arrays. Also, there should be a defined set of categories across the entire platform, and *Trivia Basket* maintainers should be able to add new categories.

User

The *User* entity represents the user of our website. A User has a *username* (unique), *password*, *fruitIcon* (their profile icon), and *iconBackgroundColor* (profile icon color).

We modeled *User* as an entity rather than an attribute, since each User should be distinct and a User has several data fields that are specific to itself.

Flashcard

The *Flashcard* entity represents the user flashcards. Flashcards are *Questions* that users have bookmarked for future review. A Flashcard has a *flashcardID* and a priority (marked by user, e.g. priority 0 for something the user has reviewed a lot and knows).

We modeled *Flashcard* as an entity rather than an attribute, since a *Flashcard* is related to both *Question* (a flashcard references a question) and *User* (a user owns flashcards). In addition, it has a "priority" data field that is specific to itself.

Game

The *Game* entity represents games on *Trivia Basket*. A Game has a *gameID* and a *gameDate* (the time that the game started).

We modeled *Game* as an entity rather than an attribute, since a *Game* is related to multiple other entities. *Game* is related to *QuestionInstances* (the questions that are displayed to users during a game), *Category* (the category settings for the game), and *User* (the players of a game). In addition, it has a *gameDate* data field that is specific to itself.

QuestionInstance

The *QuestionInstance* entity represents instances of questions displayed to users during games on *Trivia Basket*. A QuestionInstance has a *questionInstanceID*, *createdAt* (time the question was displayed user), *answeredAt* (time the question was answered), and *points* (the amount of points a user earned on the question).

We modeled *QuestionInstance* as an entity rather than an attribute, since a *QuestionInstance* is related to multiple other entities. *QuestionInstance* is related to *Question* (the question displayed), *Game* (the game that the question instance was displayed in), and *User* (the user that answered the question). In addition, *QuestionInstance* has its own data fields that are specific to itself, like *createdAt* and *answeredAt*.

Step 2: Relation Descriptions

Question - QuestionOption

Question and QuestionOption have two relations. First, a QuestionOption is an **option** for a Question, and a Question's options are QuestionOptions. This is **1 to many** relationship, since a Question has **2 or more** options, and a QuestionOption is an option for exactly **one** Question.

Next, a Question is **answered** by a QuestionOption, and a QuestionOption **answers** a Question. This relation is **one to one**, since a Question has exactly one correct answer, and a QuestionOption can only ever answer one Question.

option:

Question→QuestionOption: **2..*** . A question has two or more answer choices.

QuestionOption→Question: **1..1**. A question option is the answer choice for exactly one question.

answer:

Question→QuestionOption: **1..1** . A question has exactly one question option that is the correct answer.

QuestionOption→Question: **1..1**. A question option is the correct answer for exactly one question.

Question - Category

A Question **maps** to a Category, and a Category is **mapped** by Questions. This relation is **1 to many**, since a Question has exactly **one** category, but a Category can apply to **many** Questions.

Question→Category: **1..1** . A question has exactly one category.

Category→Question: **0..***. A category can be assigned to 0 (if a new category is created, it will have 0 questions) or more questions.

Question - Flashcard

A Flashcard **contains** a Question, and a Question **is contained by** a Flashcard. This is **1 to many** relationship, since a Question can be bookmarked by multiple users and be used in **many** flashcards, but a Flashcard references exactly **one** question.

Flashcard→Question: **1..1** . A flashcard references exactly one question.

Question→Flashcard: **0..***. A question can have multiple flashcards created from it.

Flashcard - User

A Flashcard is **owned** by a User, and a User **owns** Flashcards. This relation is **1 to many**, since a Flashcard has exactly **one** User owner, but a User can have **many** Flashcards.

Flashcard→User: **1..1** . A flashcard is owned by exactly one user.

User→Flashcard: **0..***. A user can own 0 or more flashcards.

Question - QuestionInstance

A QuestionInstance is an **instance** of a Question, and a Question is **instanced** by a QuestionInstance. This relation is **one to many**, since a QuestionInstance is the

instance of exactly one Question, but a Question can have multiple instances (the same question can be displayed in multiple games to multiple users).

QuestionInstance→Question: **1..1** . A question instance references exactly one question.

Question→QuestionInstance: **0..*** . A question can be displayed in multiple games/to multiple users, so a question can have 0 or more instances.

QuestionInstance - User

A QuestionInstance is **answered** by a User, and a User **answers** zero to many QuestionInstances. This relation is **many to one**, since a QuestionInstance is answered by one user (in a game, each user would see a copy of a question), and a User will answer multiple QuestionInstances.

QuestionInstance→User: **1..1** . A question instance is displayed to exactly one user.

User→QuestionInstance: **0..*** . A user can answer 0 or multiple question instances (e.g. playing multiple games, answering multiple questions within a game).

QuestionInstance - QuestionOption

A QuestionInstance is **answered** by a QuestionOption, and a QuestionOption is the **user answer** of a QuestionInstance. This relation is **one to many**, since a QuestionInstance has exactly one QuestionOption that represents the user's answer, but a QuestionOption can be the user answer of multiple QuestionInstances (e.g. multiple players answer the same option in the same game or across different games).

QuestionInstance→QuestionOption: **1..1** . A question instance has exactly one question option as a user answer.

QuestionOption→QuestionInstance: **0..*** . A question option can be the user answer for 0 or more question instances.

QuestionInstance - Game

A QuestionInstance is **contained** by a Game, and a Game **contains** a QuestionInstance. This relation is **one to many**, since a QuestionInstance is the instance of exactly one Game, but a Game can **contain** multiple instances (the same question can be displayed in multiple games to multiple users).

QuestionInstance→Game: **1..1** . A question instance belongs to exactly one game.

Game→QuestionInstance: **1..***. A game must have one or more question instances (questions displayed during the game).

User - Game

A User is a **player** in a Game, and a Game's players are Users. This relation is **many to many**, since a User can play in **many** games and a Game has **many** players (Users).

User→Game: **0..*** . A user can play in 0 or more games.

Game→User: **1..***. A game must have one or more players.

Game - Category

A Game is **described** by a Category, and a Category **describes** a Game. This relation is **many to many**, since a Games can select questions from multiple Categories, and a Category can be used in multiple games.

Game→Category: **1..*** . A game must have at least one category but could have several categories

Category→Game: **0..***. A category can apply to 0 (if no games use it) or any number of games.

Step 3: Project Requirements

We have seven entities (only one user entity) and have every type of relationship (one-to-one, one-to-many, and many-to-many).

Step 4: BCNF

Our database schema adheres to a BCNF form: the lefthand side of every functional dependency in any of our relations is a superkey for that relation.

User Functional dependencies:

username $\rightarrow$ *password, fruitIcon, iconBackgroundColor*

username (unique) is a superkey for User since all other attributes are in the closure of username (primary key).

Game Functional dependencies:

gameID $\rightarrow$ *gameDate*

gameID is a superkey for Game since all other attributes are in the closure of gameID.

Category Functional dependencies:

categoryID $\rightarrow$ *categoryName*

categoryName $\rightarrow$ *categoryID*

categoryID is a superkey for Category since all other attributes are in the closure of categoryID. categoryName is a superkey for Category since all other attributes are in the closure of categoryName.

GameSettings Functional dependencies:

categoryID, gameID $\rightarrow$ *categoryID, gameID*

There are no non-trivial functional dependencies in GameSettings since neither gameID or categoryID depends on the other (many to many). categoryID, gameID is a superkey for GameSettings since categoryID, gameID are the only attributes in GameSettings.

GamePlayers Functional dependencies:

username, gameID $\rightarrow$ *username, gameID*

There are no non-trivial functional dependencies in GamePlayers since neither gameID or username depends on the other (many to many). username, gameID is a

superkey for GamePlayers since username, gameId are the only attributes in GamePlayers.

Question Functional dependencies:

$questionID \rightarrow questionType, questionText, categoryID, answer$

questionID is a superkey for Question since all other attributes are in the closure of questionID.

QuestionOption Functional dependencies:

$questionOptionID \rightarrow optionLabel, optionValue, questionID$

questionOptionID is a superkey for QuestionOption since all other attributes are in the closure of questionOptionID.

QuestionInstance Functional dependencies:

$questionInstanceID \rightarrow userAnswer, createdAt, answeredAt, points, questionID, username, gameId$

$questionID, username, gameId \rightarrow questionInstanceID, userAnswer, createdAt, answeredAt, points$

questionInstanceID is a superkey for QuestionInstance since all other attributes are in the closure of questionInstanceID. questionID, username, gameId is a superkey for QuestionInstance since all other attributes are in the closure of questionID, username, gameId.

Flashcard Functional dependencies:

$flashcardID \rightarrow priority, username, questionID$

$username, questionID \rightarrow priority, flashcardID$

flashcardID is a superkey for Flashcard since all other attributes are in the closure of flashcardID.

username, questionID is a superkey for Flashcard since all other attributes are in the closure of username, questionID.

Step 5: Logical design (relational schema)

To convert our UML diagram to a relational schema for our database, we modeled one to many relationships as a foreign key reference in the "many" entity. We modeled one-to-one relationships as mutual foreign key references in both entities, and we modeled many-to-many relationships as new tables (relations) with foreign key references to the two entities. The relational schema for the database design is as follows:

User(**username**: VARCHAR(50) [PK], **password**: VARCHAR(50), **fruitIcon**: VARCHAR(50), **iconBackgroundColor**: VARCHAR(50))

Game(**gameID**: INT [PK], **gameDate**: DATETIME)

Category(**categoryID**: INT [PK], **categoryName**: VARCHAR(50))

GameSettings(**categoryID**: INT [FK to Category.categoryID], **gameID**: INT [FK to Game.gameID])

- This table represents the many-to-many relationship between Category and Game and represents the category settings of a given game.

GamePlayers (**username**: VARCHAR(50) [FK to User.uesrname], **gameID**: INT [FK to Game.gameID])

- This table represents the many-to-many relationship between User and Game and represents the players (users) in a given game.

Question(**questionID**: INT [PK], **questionType**: VARCHAR(50), **questionText**: VARCHAR(255), **categoryID**: INT [FK to Category.categoryID], **answer**: INT [FK to QuestionOption.questionOptionID])

QuestionOption(**questionOptionID**: INT [PK], **optionLabel**: VARCHAR(50), **optionValue**: VARCHAR(50), **questionID**: INT [FK to Question.questionID])

QuestionInstance(**questionInstanceID**: INT [PK], **createdAt**: DATETIME, **answeredAt**: DATETIME, **points**: INT, **questionID**: INT [FK to Question.questionID],

username: VARCHAR(50) [FK to User.username], **gameID:** INT [FK to Game.gameID], **userAnswer:** INT [FK to QuestionOption.questionOptionID])

Flashcard(**flashcardID:** INT [PK], **priority:** INT, **username:** VARCHAR(50) [FK to User.username], **questionID:** INT [FK to Question.questionID])